

Auxiliar #3

Interoperabilidad

Profesor: Nancy Hitschfeld

Auxiliares: Vicente González y Diego García

~/CC7515-1 – Computación en GPU/Aux3

> Introducción

- CUDA
- OpenCL
- Compute Shaders

~/CC7515-1 – Computación en GPU/Aux3

> CUDA

- La más conocida para GPGPU
- Consiste en **mapear** los recursos de OpenGL a CUDA
- Pipeline:
 1. Inicializar contexto OpenGL
 2. Crear los objetos de OpenGL
 3. Registrar los objetos en CUDA
 4. **Mapear** los objetos a recursos CUDA
 5. Ejecutar los kernels
 6. **Unmapear** los objetos a recursos CUDA
 7. Renderizar

Registrar

```
struct cudaGraphicsResource* resource;  
// Para un buffer  
cudaGraphicsGLRegisterBuffer(&resource, VB0, flags);  
// Para una imagen/textura  
cudaGraphicsGLRegisterImage(&resource, IMG, IMG_TYPE, flags);
```

Mappear

- Para mapear:

```
cudaGraphicsMapResources(count, resource, stream);
```

- Para unmapear

```
cudaGraphicsUnmapResources(count, resource, stream);
```

Obtener

```
void* d_data;  
cudaGraphicsResourceGetMappedPointer(&d_data, size, resource);
```


Ejemplo



<https://github.com/Seivier/NBodyCuda/tree/main>

~/CC7515-1 – Computación en GPU/Aux3

> OpenCL

- Es multi-plataforma.
- Basado en **adquirir** y **liberar** la memoria.
- Pipeline:
 1. Crear el contexto de OpenGL
 2. Crear la queue de OpenCL con el contexto de GL
 3. Crear buffers de OpenCL desde los de GL
 4. Pedir la memoria
 5. Ejecutar los kernels
 6. Liberar la memoria
 7. Renderizar

Contexto

Depende del sistema operativo:

- MacOS

```
CGLContextObj kCGLContext = CGLGetCurrentContext();  
CGLShareGroupObj kCGLShareGroup =  
CGLGetShareGroup(kCGLContext);
```

```
cl_context_properties properties[] =  
{ CL_CONTEXT_PROPERTY_USE_CGL_SHAREGROUP_APPLE,  
(cl_context_properties)kCGLShareGroup, 0};
```

- Linux

```
cl_context_properties properties[] = {  
    CL_GL_CONTEXT_KHR,  
(cl_context_properties)glXGetCurrentContext(),  
    CL_GLX_DISPLAY_KHR,  
(cl_context_properties)glXGetCurrentDisplay(),
```

```
CL_CONTEXT_PLATFORM,  
(cl_context_properties)platform, 0};
```

- Windows

```
cl_context_properties properties[] = {  
    CL_GL_CONTEXT_KHR,  
(cl_context_properties)wglGetCurrentContext(),  
    CL_WGL_HDC_KHR,  
(cl_context_properties)wglGetCurrentDC(),  
    CL_CONTEXT_PLATFORM,  
(cl_context_properties)platform, 0};
```

Registrar

```
cl_mem mem;  
mem = clCreateFromGLBuffer(context, flags, VBO, &error);
```

Manipular

- Para obtener:

```
error = clEnqueueAcquireGLObjects(queue, 1, &mem, 0, 0, 0);
```

- Para liberar:

```
error = clEnqueueReleaseGLObjects(queue, 1, &mem, 0, 0, 0);
```

Ejemplo



<https://github.com/Seivier/NBodyCuda/tree/main>

en `example/opengl`

~/CC7515-1 – Computación en GPU/Aux3

> Compute Shaders

Compute Shaders

- Disponible a partir de OpenGL 4.3.
- Vive en un pipeline independiente a todo.
- Necesita su propio programa.
- Pipeline:
 1. Crear el contexto de OpenGL.
 2. Compilar el compute shader.
 3. Ejecutar el shader (*dispatch*).
 4. Usar barreras para esperar a que termine.
 5. Renderizar

Compute Shader

```
#version 430 core
// el localSize o blockSize
layout (local_size_x = 1, local_size_y = 1, local_size_z = 1) in;

// buffers
layout (std430, binding = 0) buffer particles {
    ...
};

// vars
layout (location = 0) uniform uint size;

void main() {
    uint index = gl_GlobalInvocationID.x; // global id
    // ...
}
```

Compute Shader

- Para obtener la cantidad de *work groups* (bloques):

```
uvec3 n = gl_NumWorkGroups
```

- Para obtener el tamaño de los *work groups*:

```
uvec3 size = gl_WorkGroupSize
```

- Para obtener el ID del *work group* actual:

```
uvec3 blockId = gl_WorkGroupID
```

- Para obtener el ID local del *work item* (thread):

```
uvec3 lId = gl_LocalInvocationID
```

- Para obtener el ID global del *work item* (thread):

```
uvec3 gId = gl_GlobalInvocationID
```

Shader Storage Buffer Object

- Introducido junto a los Compute Shaders.
- Sirve para almacenar datos arbitrarios en un buffer en la GPU.
- Se inicializa de la misma manera que cualquier buffer:

```
unsigned int ssbo;  
glGenBuffers(1, &ssbo);  
glBindBuffer(GL_SHADER_STORAGE_BUFFER, ssbo);  
glBufferData(GL_SHADER_STORAGE_BUFFER, sizeof(data), data, GL_DYNAMIC_DRAW);
```

- Además de bindear a OpenGL, se debe bindear al *layout* del shader:

```
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, ssbo);  
// Donde 0 es el binding dentro del layout del shader
```

Shader Storage Buffer Object

- Introducido junto a los Compute Shaders.
- También podemos acceder al buffer dentro de otros shaders.
- Ejemplo:

```
#version 430 core
// in y out...

// ssbo
layout (std430, binding = 0) buffer dataBuff {
    float data[];
};

// uniforms...

void main() {
    float x = data[3*gl_VertexID];
    float y = data[3*gl_VertexID+1];
    float z = data[3*gl_VertexID+2];
    gl_Position = vec4(x, y, z, 1.0f); // sin vbo!!!!
}
```

Para acceder en el vertex shader, usamos un EBO y, aunque no lo usemos, necesitamos crear un VBO (vacío).

Ejecución

```
// shader programs
unsigned int compute;
unsigned int render;

// ...

while(...) {
    glUseProgram(compute);
    glBindBuffer(GL_SHADER_STORAGE, ssbo);
    glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 0, ssbo);
    // ejecucion
    glDispatchCompute(globalX, globalY, globalZ); // num de workgroups

    // sincronizacion
    glMemoryBarrier(GL_ALL_BARRIER_BITS); // podemos ser menos exigentes
    glUseProgram(render);

    // renderizado
}
```

Ejemplo



<https://github.com/Seivier/NBodyCuda/tree/main>

en `example/compute`

~/CC7515-1 – Computación en GPU/Aux3

> Referencias

- CUDA:

https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__OPENGL.html#group__CUDART__OPENGL

- OpenCL (macOS):

https://developer.apple.com/library/archive/documentation/Performance/Conceptual/OpenCL_MacProgGuide/shareGroups/shareGroups.html

- Compute Shaders:

<https://learnopengl.com/Guest-Articles/2022/Compute-Shaders/Introduction>

Para ver más shaders avanzados puede revisar el libro “OpenGL 4.0 Shading Language Cookbook”.